Django Apps & the Model Registry

CrossTwin · architecture

core/utils.py::build_model_registry() scans nine domain apps; every generic endpoint is driven by the registries

Django · Vector · Raster · WMS

DOMAIN APPS (ALLOWED_APPS) — MODELS · CALCULATIONS.PY · VIEWS

administrative (5)
Province › City › District › Neighborhood
PopulationProjection

physicalEnv (17)
LandCover · DEM · DSM
soil map · groundwater
HILUCS · soil.py (SCS)
svf.py · UrbanFabricCut

watersupply (17)
extraction · treatment
pipes · NRW · OPEX
SupplySecurity

housing (9)
supply/demand · HPI
mortgage · rentals
affordability
social profile (KEA)

urban_heat (12)
UTCI · PET · Tmrt
LST · SVF · SUHII
NBS · WBGT · lst/ SMW
heat per buurt (KEA)

builtup (7)
ZoningArea (HILUCS)
Street · Park · Facility
Building · Property
climate risk (KEA)

nature (7)
green / blue
layers · trees

weather (7)
WMS layers
time steps

Energy (3)
energy labels

Number tags = model classes per app. Indicator dashboards (watersupply, housing, urban_heat, builtup) follow calculations.py → views._build_indicators() → HTMX partial; MOCK_DATA fills in when a province is missing.

`DAG edges documented in calculations.py docstrings → core/DAG.dot (DPSIR)`

scan models

build_model_registry()
core/utils.py — scans allowed_apps at startup

has GeometryField, no RasterField → VECTOR
has RasterField → RASTER · 'WMS' in key → WMS

MODEL_REGISTRY
all models

VECTOR_REGISTRY
GeoJSON, bounds, editing

RASTER_REGISTRY
COG export, tile URLs

WMS_REGISTRY
WMS layer catalog

importer
field mapping, upsert keys

mainMap
/api/layers/, GeoJSON, editing

core.signals
post_save → COG, cache bump

weather
WMS catalog, tile proxy